

Behavior Anomaly Detection System — Code Explained

A plain-language walkthrough of every folder, every file,
and what the code actually does

Hybrid SIEM + Network + Host detection with Machine Learning
PMICS Thesis Project · University of Dhaka

Contents

Contents	3
1. What this project is, in plain words	4
The problem	4
What this software does	4
Why three layers instead of one	4
An important design rule	4
2. The words you need (mini glossary)	6
3. How the system works, end to end	7
4. The three detection layers	9
5. The three data planes (host, network, SIEM)	10
6. Folder map — what each folder is for	11
7. File-by-file walkthrough	13
Root folder — the front door	13
config/ — the settings	14
core/ — the brain	14
collectors/ — the eyes	16
feature_engine/ — the translator (events to numbers)	18
behavior_detection/ — the judges	19
ml/ — the machine learning	20
models/ — the shared vocabulary	20
specs/ — the knowledge base	21
reporting/ — the output a human reads	22
readers/ and utils/ — two small helpers	22
scripts/ — standalone tools	22
The data and output folders	23
8. How to run the software	25
The easiest way	25
From the command line	25
On Linux and macOS	25
What you will see	25
9. Where the numbers come from (results)	26
The sample demonstration (100 users)	26
Against real recorded attacks	26
The academic benchmark	26
An honest note on the demonstration figures	27
In summary	28

Contents

This document explains the whole project to someone who has never written a line of code. It is organised in three passes: first the **idea**, then the **journey of a single piece of evidence** through the system, and finally a **tour of every folder and file** with the logic inside each one described step by step.

- 1. What this project is, in plain words
- 2. The words you need (mini glossary)
- 3. How the system works, end to end
- 4. The three detection layers
- 5. The three data planes (host, network, SIEM)
- 6. Folder map — what each folder is for
- 7. File-by-file walkthrough (the main part)
- 8. How to run the software
- 9. Where the numbers come from (results)

1. What this project is, in plain words

The problem

Every computer keeps a diary. Each time someone logs in, opens a file, plugs in a USB stick, or starts a program, the computer writes a line in that diary. On a busy machine this is thousands of lines a day — far too many for a human to read.

Attackers hide inside this flood. The dangerous ones do not announce themselves; they look almost exactly like ordinary work. Two kinds are especially hard to catch, and they are the focus of this project:

- **U2R (User-to-Root)** — someone who already has a normal account quietly grants themselves administrator power.
- **R2L (Remote-to-Local)** — an outsider guesses or steals a password and walks in as if they belonged.

Classic security tools catch what they have seen before, because they work from a list of known bad things. They are poor at the rare and the new.

What this software does

This software reads the computer's diary automatically and answers three questions about every user session:

- **Did anything *known-bad* happen?** — for example a command that hides itself, a program run straight off a USB stick, or a contact with a known criminal server. This is the **rules** layer.
- **Do several small things add up to an attack story?** — a USB went in, then an unsigned program ran, then it phoned a stranger. Each step alone is weak; together they are damning. This is the **correlation** layer.
- **Is this person behaving unlike themselves?** — normally logs in at 9 a.m. and touches ten files; today logged in at 2 a.m. and touched two hundred. Nothing on any blacklist, but clearly odd. This is the **machine-learning** layer.

It then combines the three answers into one verdict — **SAFE, REVIEW, SUSPICIOUS** or **CRITICAL** — writes an alert if needed, and draws a dashboard a human can read in seconds.

Why three layers instead of one

Because each covers the other's blind spot. Rules are precise but only know yesterday's attacks. Machine learning notices anything unusual but cannot say what it means. Correlation turns scattered hints into a narrative. Used together they catch more, and — just as important — they explain *why*, which a human investigator needs.

An important design rule

The system never calls a program dangerous just because of its name. Opening PowerShell is not an attack; opening PowerShell with a hidden, scrambled command that downloads a file from the internet is. The software always looks at the **context**: the full command typed, which program

launched it, where the file came from, whether it is digitally signed, and what happened around it. This is the same approach real commercial security products take, and it is the main reason the false-alarm rate stays low.

2. The words you need (mini glossary)

These few words appear throughout the code. Nothing else is assumed.

Term	What it means here
Process	A running program. Opening Chrome starts a Chrome process.
Command line	The full text used to start a program, including its options. This is where intent hides.
Parent process	The program that started another one. Word starting PowerShell is suspicious; you clicking PowerShell is not.
Event	One line of the computer's diary: who, what, when, where.
Event log	The diary itself. Windows calls it the Security log; Linux uses <code>/var/log/auth.log</code> ; macOS uses the unified log.
Baseline	A person's normal habits, learned from past activity.
Feature	One measurable number about a session — e.g. login hour, or how many files were touched. The model only understands numbers.
Isolation Forest	The machine-learning method used. It learns what normal looks like and flags points that stand apart.
Correlation	Joining several separate events into one attack story.
MITRE ATT&CK	A worldwide catalogue of attacker techniques, each with a code such as T1059. Used so findings are standard.
Suricata	A separate tool that watches network traffic.
Wazuh	A separate tool (a SIEM) that watches computer logs.
False positive	A false alarm — normal activity reported as an attack.

3. How the system works, end to end

Follow one piece of evidence — say, a USB stick being plugged in at 2 a.m. — through the whole machine. Every part of the code exists to serve one of these seven steps.

Step 1 — Collect

A **collector** reads the computer's diary. There is one collector per operating system (Windows, Ubuntu, macOS), plus one that reads network reports from Suricata and one that reads SIEM reports from Wazuh. Their only job is to fetch raw records.

Step 2 — Normalise

A **parser** converts each raw record — whichever operating system it came from — into one common shape called an **Event**: when, who, what kind, from where, and the details. After this step the rest of the system no longer cares which operating system it came from. This is why the project works on three platforms without three copies of the logic.

Step 3 — Detect (add meaning)

A **detector** reads each Event and asks: does this mean anything bad? It examines the command line, the parent program, the file's location, its signature. When it recognises something, it writes an extra Event describing the *meaning* — for example `USB_EXECUTABLE_RUN` or `ENCODED_COMMAND` — and attaches the matching MITRE code. The original evidence is kept alongside.

Step 4 — Summarise into numbers

The **feature engine** gathers all of one person's events for the session and boils them down to a fixed list of numbers: login hour, number of files touched, was a USB used, did an encoded command run, and so on. This numeric summary is called a **feature vector**. Machine learning can only work with numbers, so this step is the bridge between evidence and mathematics.

Step 5 — Judge, three ways

The same feature vector is now handed to three independent judges:

- The **risk engine** adds up weights for known-bad things it sees.
- The **correlation engine** looks for combinations that tell an attack story.
- The **machine-learning model** compares the behaviour with learned normal and reports how unusual it is.

Step 6 — Decide

The **decision engine** merges the three opinions into one verdict. A confirmed attack story outranks everything; a high risk score plus an unusual pattern is critical; an unusual pattern alone only asks a human to review.

Step 7 — Report

If the verdict is serious, the **alert manager** writes an alert containing the verdict, the reasons, the MITRE codes, and — importantly — the actual offending commands and file names. The **dashboard** then renders everything as a web page a human can scan at a glance.

In one line: collect → normalise → add meaning → turn into numbers → judge three ways → decide → report.

4. The three detection layers

Layer	What it is good at / not good at
Rules (signatures)	Recognises known-bad patterns in commands, paths and files — encoded PowerShell, certutil downloads, ransomware wiping backups, credential theft. Precise and explainable, but only knows what it has been taught.
Correlation	Joins several weak signals into a strong, named attack chain (e.g. 'USB Malware Activity'). Turns dots into a picture and raises confidence enough to justify a CRITICAL verdict.
Machine learning (Isolation Forest)	Learns each machine's normal behaviour and flags deviation — odd hours, unusual volume, new devices, many source addresses. Catches the new and the rare that no rule describes, but cannot name the threat; it only says 'this is unlike normal'.

The thesis argument rests on this pairing: rule-based tools miss rare U2R/R2L attacks, and pure anomaly detection cannot explain itself. Together, with correlation in the middle, each covers the other's gap.

5. The three data planes (host, network, SIEM)

The software does not only look inside the computer. It reads three independent viewpoints, and the most valuable findings come from agreement between them.

Plane	What it sees
Host (our own collectors)	Inside the machine: programs started and their full commands, files touched, logins, USB devices, digital signatures.
Network (Suricata)	On the wire: port scans, contact with known criminal servers, large outbound transfers (data theft), network attack signatures.
SIEM (Wazuh)	A second opinion from a professional log-analysis product: brute force, privilege escalation, rootkits — already mapped to MITRE.

Cross-plane correlation is the project's distinctive contribution. A scrambled command on the host is suspicious; the same machine also phoning a known criminal server is a *confirmed* command-and-control channel. Neither plane alone can say that.

Note. Important: Suricata and Wazuh are separate products. This software does not launch them — it *reads the reports they write*. In the demo, sample report files stand in for them; in a real deployment the file paths in the settings point at the live tools.

6. Folder map — what each folder is for

Each folder holds one job. If you remember this table, you can find anything in the project.

Folder	Its one job
(root)	The front door: <code>bad.py</code> is the program you run.
<code>config/</code>	Settings — file paths and switches, all in one place.
<code>core/</code>	The brain: the pipeline that runs everything, the four detectors that add meaning, the database, the Event shape.
<code>collectors/</code>	The eyes: one reader per data source (Windows, Ubuntu, macOS, Suricata, Wazuh) plus the parsers that normalise their raw records.
<code>feature_engine/</code>	The translator: turns a pile of events into the fixed list of numbers the model needs; also learns baselines and builds training data.
<code>behavior_detection/</code>	The judges: risk scoring, correlation chains, the final decision, and alert writing.
<code>ml/</code>	The machine-learning model: training it, using it, scoring it.
<code>models/</code>	The definition of a feature vector — the shared vocabulary of numbers.
<code>specs/</code>	The knowledge base: detection rules, MITRE mappings and the allowlist. Plain lists a human can edit without touching logic.
<code>reporting/</code>	The output: the HTML dashboard and the charts.
<code>readers/</code>	A helper to read Windows <code>.evtx</code> evidence files.
<code>utils/</code>	A helper to load sample events from a JSON file.
<code>scripts/</code>	Standalone tools: generate sample users, replay real attacks, run the NSL-KDD benchmark, build this document.
<code>demo/</code>	The reproducible demo data: 100 sample users.

data/	Datasets (real attack samples, NSL-KDD) and the sample network / SIEM / macOS reports.
software/	The runnable demo for each operating system.
reports/, alerts/, logs/, database/	Outputs produced while running: dashboard, alerts, logs, and the small database of events and baselines.

7. File-by-file walkthrough

Now the tour. For every file you get: **what it does**, and **how it works** — the logic inside, step by step, in ordinary language. Small files are covered almost line by line; large files are covered block by block, because that is how they are actually organised.

Root folder — the front door

bad.py — the program you run

This is the entry point. Everything starts here. 'bad.py' stands for Behavior Anomaly Detection System.

- **Find the project first.** Before doing anything, it walks up from its own location looking for the folders `demo/` and `config/`. When it finds them it makes that the working folder. This is why the program works whether you double-click it, run it from `software/`, or call it from anywhere — it always finds its data.
- **Offer a menu when double-clicked.** If started with no instructions (what happens on a double-click), it prints a menu — [1] Own, [2] Analyze, [3] Live — reads your choice, and keeps the window open at the end so you can read the output.
- **Understand typed commands.** Otherwise it reads the command you typed (`analyze`, `own`, `watch`, `learn`, `evaluate`, `benchmark`, `dashboard`, `demo`) and runs the matching function.
- **Ask for Administrator when needed.** The 'own' and 'watch' commands must read the Windows Security log, which requires Administrator rights. If it is not elevated, it re-launches itself through the Windows permission prompt and lets the new window do the scan.
- **Each command is a short recipe** composed of pipeline steps: 'analyze' = make sure a model exists, load the sample session, judge it, draw the dashboard. 'own' = the same but with a live scan of this machine, and it also learns this machine's normal on the first run.

Note. There is no development/production switch anywhere. What the program does is decided by *which command you choose*, not by a hidden setting.

main.py — an old door left open

Seventeen lines. It exists only so that anyone used to typing `python main.py` still gets something sensible: it simply calls `bad.py` with the 'demo' command.

Run-Demo.bat — the double-click launcher (Windows)

A tiny Windows batch file. It moves into the project folder and starts `software\windows-demo.exe`. Double-clicking it is the easiest way to run the demo.

requirements.txt — the shopping list

Lists the outside libraries the project needs (scikit-learn for machine learning, pandas for tables, matplotlib for charts, and the Windows-only helpers). 'pip install -r requirements.txt' installs them all.

README.md, BUILD.md — the instructions

README explains the project and how to run it; BUILD explains how to package it into a standalone program for each operating system.

config/ — the settings

settings.py — every path and switch in one place

A short list of names and values, so nothing is hidden inside the code. It says where the sample users live, where the datasets live, where Suricata and Wazuh reports are read from, where reports and alerts are written, and how many normal samples are needed before the model can be trained.

- **LEARNING_MODE** — when on, sessions judged safe are remembered as more examples of normal.
- **SURICATA_EVE_FILE / WAZUH_ALERTS_FILE** — change these two lines and the software reads a real Suricata or Wazuh installation instead of the samples. That is the whole integration.

core/ — the brain

event.py — the common shape

Twenty-five lines, and arguably the most important idea in the project. It defines what an Event is: timestamp, username, operating system, event type, source, IP address, details, and a unique record id. Every collector, on every operating system, must produce this same shape. Because of that single agreement, the rest of the system is written once and works everywhere.

database.py — the memory

A small SQLite database (a single file on disk) with three tables: events, baseline (each user's learned normal) and users.

- **create_tables** makes the tables if they do not exist.
- **insert_event** stores one event; if the same record id already exists it is silently skipped, so re-reading a log never creates duplicates.
- **insert_events_bulk** stores thousands of events in one go. This matters: loading the 100-user demo went from over four minutes to about seventeen seconds after this was added.
- **get_events_by_user** fetches one person's events — used to pull the actual offending commands out for the alert.
- **get_usb_drive_letters** remembers which drive letters were USB sticks, so a program run from E:\ can later be recognised as having come from removable media.

logger.py — the diary of the program itself

Sends detailed running commentary to logs/bads.log and keeps the screen clean — only warnings and errors appear there. Before this, every database write printed to the screen, which buried the actual results and made the program look broken.

process_detector.py — the main detective

Given a program's path, its full command line, its parent program and the list of USB drives, it decides what the program was really doing. This is where the project's central rule lives: never judge by name alone, always by context.

- **First, the allowlist.** If the command is on the trusted list (the security tool's own work, or something the operator marked as known-good), stop immediately and report nothing.

- **Then keyword rules.** It searches the command for every known pattern from `specs/process_rules.py` — scrambled commands, downloads, hidden windows, privilege grabs, scanners, reverse shells, ransomware backup-wiping, credential theft. Each match adds a weight and records one meaning plus its MITRE code.
- **Then the path.** Running from a temporary or downloads folder adds suspicion.
- **Then removable media.** If the program's path starts with a drive letter known to be a USB stick (or a Linux/macOS removable mount), it reports 'ran from USB'.
- **Then the file name.** `invoice.pdf.exe` — a harmless-looking extension followed by a real one — is the classic disguise, and is reported as masquerading.
- **Then the parent.** If Word or Outlook started PowerShell, that is the classic phishing-document chain and is reported as such.
- **Finally the 'concern' tier.** If none of that matched but the program is simply an interpreter such as PowerShell, it records a low-severity note worth ten points. Noticed, not accused.

Note. Because the same class is used for Windows, Linux and macOS commands, one set of rules protects all three platforms.

file_detector.py — the detective for files

Given a file that was just created and the program that created it, decides whether it is a suspicious download. The rule is the same philosophy: creating a file is never suspicious by itself; a *program or script* landing in a *downloads, temp, desktop or USB* location is. It also spots the double-extension disguise.

network_detector.py — the detective for network reports

Reads one line of Suricata's report and turns it into meaning. Suricata already classifies its alerts, so this file mostly translates that classification into the project's own vocabulary: scan categories become 'port scan', trojan and command-and-control categories become 'malicious host contact', information-leak categories become 'data exfiltration'. Severity scales the weight.

- It also reads completed connections: a very large upload to an address outside the local network is reported as possible data theft even when no alert fired.

wazuh_detector.py — the detective for SIEM reports

Reads one Wazuh alert. Ignores the routine ones (below level 5). For the rest it records a general 'SIEM alert', and — where Wazuh names a pattern it already understands, such as brute force or privilege escalation — it reinforces the project's own matching signal. It also reuses the MITRE codes Wazuh has already worked out.

signature_checker.py — is this program genuine?

Asks Windows whether a program is digitally signed by a real publisher. Because this check is slow, it only runs for programs that already look suspicious (from a temp folder, from USB, or with a disguised name), and the answer is remembered. If the answer cannot be determined, the program is **never** assumed to be unsigned — guessing would create false accusations.

pipeline.py — the conductor

The largest and most important file in `core/`. Every command in `bads.py` is built from these steps, so the logic exists once.

- **get_os_collector** — looks at which operating system is running and hands back the matching collector.
- **learn_baseline** — clears the events table, loads a body of normal behaviour, turns it into feature vectors, saves each user's baseline, and trains the model.
- **collect_live** — runs the real collectors on this machine. It warns if not running as Administrator, and if the host collector fails it carries on rather than crashing.
- **load_session_file** — the reproducible alternative: load a sample session from a file, optionally joined with sample Suricata and Wazuh reports credited to a chosen user, so the cross-plane chains can be demonstrated identically every time.
- **analyze** — the heart. For each user: fetch the baseline, compare, look for correlation chains, add up risk, ask the model, decide, print, and write an alert with evidence when serious. If a user has no baseline yet it still runs the threat checks — it simply skips the 'unlike you' comparison. (Before this fix, new users were skipped entirely and the live scan appeared to do nothing.)
- **indicators_for_user** — pulls the real offending commands, file names and network signatures out of the database so the alert shows the evidence, not just a verdict.
- **establish_baselines / learn_own_normal** — on a machine's first scan, remember its normal, and teach a *separate* model that normal. The separation matters: it keeps live learning from ever corrupting the reproducible sample model.
- **evaluate / build_dashboard** — score the model, and draw the dashboard (opening it in a browser when asked).

collectors/ — the eyes

One reader per data source. They differ inside because each operating system keeps its diary differently — but they all hand back the same Event shape, so the rest of the system never notices the difference.

base_collector.py — the shared skeleton

Forty lines. Says every collector must have a `collect()` method, and gives them all the same start/stop/info/warning messages. It is a contract, not logic.

windows_collector.py — the Windows reader (the biggest collector)

Reads the Windows Security log through the official Windows interface.

- **Remembers where it stopped.** It stores the last record number it read in a small file, so each run only processes new activity.
- **Collects logins** (event 4624 success / 4625 failure), skipping the noisy service logons.
- **Collects program starts** (event 4688), which include the full command line and the parent program. It ignores a list of ordinary Windows system programs so they do not drown the results, then sends each remaining one to the ProcessDetector and stores whatever meaning comes back.
- **Collects file access** (event 4663) and passes file names to the FileDetector.
- **Collects file creation** from Sysmon if it is installed; if not, it simply says so and moves on.
- **Finds USB drives** — asks Windows which drive letters are removable right now, and adds any remembered from past insertions, so the detector can recognise a program run from a USB stick.

- **Watches for USB insertions** for a few seconds through the USB monitor.
- **Never crashes the scan.** Each of those phases is wrapped so that a permission error prints a clear message ('needs Administrator') and the run continues. Before this, a non-Administrator double-click died silently after the first line.

windows_usb_monitor.py — the USB watcher

Waits for Windows to announce that a disk was plugged in or removed, checks it really is USB, and records it. It is time-boxed to a few seconds so a single scan can never hang forever.

ubuntu_collector.py — the Linux reader

Reads /var/log/auth.log line by line, remembering its byte position so each run only reads new lines (and starting over if the file was rotated). Login lines become login events; sudo command lines become program-start events and are sent to the ProcessDetector. If auditd is installed it also reads program executions and newly created files from there; if not, it says so and carries on.

mac_collector.py — the macOS reader

Two ways in: on a real Mac it asks the unified log for recent activity; otherwise it reads a sample export file (used for the demo and for testing on other machines). Either way each record becomes an Event and program records go to the ProcessDetector — which is why macOS-specific attacks (AppleScript shells, programs run from /Volumes, launchctl persistence) are caught by the same rules.

suricata_collector.py — the network reader

Suricata writes one JSON line per network event to a file called eve.json. This collector reads that file from where it last stopped, hands each line to the NetworkDetector, and stores the resulting network events. A 'reset' option re-reads the whole file from the start, which is what the reproducible demo uses.

wazuh_collector.py — the SIEM reader

Exactly the same pattern for Wazuh's alerts.json. Each alert goes to the WazuhDetector and the results are stored. Because these events carry the same shape as everything else, a Wazuh finding can take part in a correlation chain alongside a host or network finding.

collectors/parsers/ — the normalisers

Small, single-purpose translators. Each one takes a raw record and returns a clean Event.

- **base_parser.py** — the contract: every parser has a parse().
- **login_parser.py** — reads a Windows login record, decides success or failure from the event number, pulls out the user and the source IP address.
- **process_parser.py** — reads a Windows program-start record and keeps the three things that matter: the program's path, the full command line, and the parent program.
- **file_parser.py** — reads a Windows file-access record: which file, which program, what kind of access.
- **usb_parser.py** — describes a USB disk, and works out its drive letter so later program runs from that letter can be linked to it.

- **ubuntu_parser.py** — understands the shapes of auth.log lines (SSH accepted, SSH failed, session opened/closed, sudo commands) using text patterns, and gives each line a stable id so re-reading the log never duplicates events.
- **mac_parser.py** — turns a macOS record into an Event, packing the program, command and parent into the same layout the Windows records use so the detector treats them identically.

feature_engine/ — the translator (events to numbers)

aggregator.py — sort the pile by person

Sixteen lines. Takes all the events and groups them per username, because the system judges one *person's session* at a time.

extractor.py — the heart of the translation

Takes one person's events and produces the fixed list of numbers (the feature vector). It walks through the events once, counting.

- **Counts each kind of meaning** — how many encoded commands, USB program runs, privilege escalations, malicious host contacts, and so on. Each becomes one number.
- **Notes the timing** — average login and logout hour.
- **Builds the behavioural statistics** the model uses: how long the session lasted, how spread out it was, what share of activity was outside office hours, weekend activity, how intense it was (events per active hour), how many *different* programs were run (variety, not just volume), the balance of file work to program work, how many login attempts, USB use, and how many different network addresses appeared.

These behavioural numbers are deliberately chosen so that each one asks a *different* question about the person. That is what lets the model notice 'unlike you' rather than merely repeating what the rules already found.

baseline.py — remembering a person's normal

Saves and loads each user's baseline (their usual login hour, usual amount of program and file activity, usual USB use) in the database.

comparator.py — how different is today?

Compares this session against that person's baseline, feature by feature, and marks each as normal or changed.

- For hours and counts of failures it uses a fixed tolerance.
- For busy numbers such as programs started and files touched it uses a **proportional** tolerance (about half the baseline, with a floor). This matters: a real workstation runs hundreds of programs and the count swings wildly between scans. With the original fixed tolerance of one, every single real scan looked 'changed'. The proportional rule is what makes the baseline usable on a real machine.

dataset_builder.py — writing training rows

Appends one feature vector as a row in the training spreadsheet (a CSV). It also checks the column headings: if the list of features has changed since the file was written, it starts the file over rather than

silently writing values into the wrong columns.

dataset_generator.py — the cold-start helper

A brand-new installation has no history, and the model cannot learn 'normal' from nothing. This file takes a real baseline and creates many slightly-varied copies of it as synthetic normal examples, plus obviously-malicious examples used only for evaluation. Once enough real history exists, this is no longer used.

behavior_detection/ — the judges

risk_engine.py — adding up the danger

Produces a number and a list of human-readable reasons. It works in two tiers, and the split is deliberate.

- **Tier 1 — deviation.** Small weights for simply being different from your baseline (unusual hour, more activity). A change of routine is worth noticing, not accusing.
- **Tier 2 — context.** Large weights for genuinely dangerous things: an encoded command, a program run from USB, contact with a criminal server, wiping backups, stealing credentials. These are threats regardless of anyone's habits.

This encodes the project's policy in one sentence: *a user running PowerShell is a concern; a user running a harmful command is a threat.*

correlation_engine.py — turning dots into stories

The most 'thinking' part of the system. It looks at the whole feature vector for combinations that mean something together, and names them.

- **Host chains** — USB inserted plus a program run from it; a disguised or unsigned program from USB; Word starting PowerShell that then downloads; encoded PowerShell that downloads and runs; trusted Windows tools abused to fetch code; ransomware wiping backups; credential theft; a reverse shell after a payload arrived; persistence installed alongside a payload.
- **Cross-plane chains — the project's distinctive part.** These only fire when two *independent* viewpoints agree: an encoded command on the host *and* a malicious server contact on the network is a confirmed command-and-control channel; a network scan *and* host login failures is a confirmed remote break-in attempt; credential theft *and* a large upload is confirmed data theft; a Wazuh SIEM alert *and* an independent behavioural finding is a hybrid confirmation.

Each chain carries a severity, an attack class (U2R, R2L, execution, exfiltration...), the MITRE codes, and a sentence explaining it to the analyst.

decision_engine.py — the final word

Forty-three lines that merge the three opinions:

- A CRITICAL correlation chain wins immediately — a confirmed story beats any score.
- Otherwise: high risk *and* an unusual pattern is CRITICAL; high risk alone is SUSPICIOUS; low risk and normal behaviour is SAFE; an unusual pattern alone only asks a human to REVIEW.

analyzer.py — naming the score

Eleven lines: turns a risk number into LOW, MEDIUM or HIGH.

alert_manager.py — writing the alert

When the verdict is serious it writes the alert twice: as a detailed JSON file (one per alert, for records) and as one line appended to a rolling spreadsheet that the dashboard reads. The alert carries the verdict, the score, the reasons, the MITRE codes, the machine-learning score, which behaviours deviated, and the **actual offending commands and file names** — so an investigator sees the evidence, not just a conclusion.

ml/ — the machine learning

trainer.py — teaching the model

Reads the training spreadsheet, keeps only the rows marked normal (the model must never be shown an attack as 'normal'), refuses to train if there are too few examples, and builds a two-step model:

- **Step one — standardise.** The features have wildly different scales: a login hour is 0–23, a file count can be 300, and a threat flag is 0 or 1. Without this step the big numbers drown the small ones and the model barely notices the threat flags. Standardising puts them all on the same footing.
- **Step two — Isolation Forest.** Three hundred random decision trees learn the shape of normal. Anything that can be separated from the crowd in very few splits is unusual.

Both steps are saved together, so predictions always get the same treatment as the training data.

predictor.py — asking the model

Loads the saved model and answers two questions about a session:

- **Normal or anomaly**, plus an **anomaly score from 0 to 100** where 50 is the boundary and higher means more unusual. (The earlier version reported a confusing 'confidence' where an attack showed 45%.)
- **Which behaviours deviated** — it reuses the model's own memory of what normal looks like to work out, for each behavioural feature, how many standard deviations this session is away. That is how the dashboard can say 'login time 2:00 vs usual 9:30, that is 13 standard deviations away'. The explanation is the model's own reasoning, not a separate guess.

evaluator.py — marking the model's exam

Runs the model over a held-out set it never trained on, and reports accuracy, precision, recall and F1, plus the confusion matrix. It also produces a continuous score for every sample so an ROC curve can be drawn, then hands everything to the report generator.

model.joblib / model_own.joblib — the trained brains

Two saved models, kept deliberately separate: one learned from the sample users (used by Analyze, so the demo is always reproducible), and one learned from this actual machine (used by Own and Live). Keeping them apart fixed a real bug — live learning had been polluting the sample model until a genuine attack stopped being flagged.

models/ — the shared vocabulary

feature_vector.py — the list of numbers

Defines every feature the system measures and, crucially, two lists:

- **ML_FEATURES** — everything recorded about a session (about fifty numbers), used as the spreadsheet columns.
- **ML_MODEL_FEATURES** — the eighteen *behavioural* features actually shown to the machine-learning model, in five groups: **timing** (login hour, logout hour, session length, off-hours share, weekend activity); **intensity** (activity rate, active-hours spread, program volume, file volume); **style** (program variety, file-vs-process balance, file-type variety); **authentication** (failed logins, login attempts, off-hours logins); and **device / network** (USB use, removable-media activity, network sources). Each asks a different question — none is a restatement of another. Every one is collectable live on a real endpoint, so the same eighteen drive both the benchmark and live monitoring.

Keeping these two lists in one file is why a new feature can never drift out of step between training, prediction and evaluation.

specs/ — the knowledge base

These files contain no logic — only lists a security analyst can read and edit. The logic that uses them lives in `core/`. This separation is deliberate: knowledge changes far more often than code.

process_rules.py — what a dangerous command looks like

The largest knowledge file. Groups of patterns, each with a weight and the meaning it produces: PowerShell abuse (scrambled commands, policy bypass, hidden windows, downloads), trusted Windows tools abused to fetch code, privilege grabs on Windows and Linux, scanners and password crackers, reverse shells, ransomware wiping backups, credential and secret theft, persistence. It also lists temp and download locations, removable-media mounts, the document programs that should never start a shell, and the extensions used for disguise.

mitre_rules.py — the standard names

Maps every pattern to its MITRE ATT&CK technique, so a finding reads 'T1059.001 PowerShell' rather than a private label. This is what makes the output comparable with the rest of the industry.

network_rules.py — reading Suricata's language

Maps Suricata's own alert categories to the project's meanings, scales weight by severity, and sets the threshold above which an upload counts as possible data theft (plus the list of private addresses that never count as 'leaving the network').

wazuh_rules.py — reading Wazuh's language

Maps Wazuh rule groups to the project's meanings, and converts Wazuh's 0–15 severity into a weight. Alerts below level 5 are ignored as routine.

exclusions.py — the allowlist

The trusted list. Anything matching it is dropped before any judgement. It ships with the security tool's own operations (so the monitor never accuses itself of the work it is doing) and an empty space for the operator to add known-good administrative or automation commands. This mirrors how commercial products handle the same problem, and it lowers false alarms without weakening

detection: the same technique used by anything *not* on the list is still caught.

reporting/ — the output a human reads

report_generator.py — the charts and the numbers

Turns one evaluation into three files: metrics.json (all the scores), confusion_matrix.png (a labelled grid of right and wrong answers) and roc_curve.png (how well the model separates attack from normal at every possible threshold, summarised by the area under the curve). It draws without opening any window, so it works on a server.

dashboard.py — the single page

Builds one self-contained HTML file — no internet, no server needed. Top to bottom it shows:

- **Behavioral Anomaly Detection** — the headline. One card per scanned session with a coloured meter for the anomaly score and, beneath it, exactly which behaviours deviated and by how much. It shows every session, not only alerts, so live monitoring is visibly working even when everything is clean.
- **Model performance** — accuracy, precision, recall, F1, ROC area.
- **Charts** — the confusion matrix and ROC curve, embedded directly into the page.
- **Recent alerts** — the serious findings, each with the MITRE codes and, underneath, the actual offending commands and file names.
- In live mode it adds a LIVE badge and refreshes itself, so new findings appear without touching the browser.

readers/ and utils/ — two small helpers

readers/evtx_reader.py — opening Windows evidence files

Twenty-three lines. Opens a Windows .evtx evidence file and hands back its records. Used to replay real, publicly published attack recordings through the system.

utils/data_loader.py — loading sample events

Thirty-one lines. Reads a JSON file of events and converts each entry into the standard Event shape.

scripts/ — standalone tools

generate_sample_users.py — building the demo population

Creates the reproducible demo: 100 users. Ninety-seven are 'old' users with a history — each given their own believable habits (a personal login hour, activity level, whether they use USB sticks, their own address) that stay consistent between their history and their current session, so they do not raise false alarms. Three are brand-new users who appear today with no history, to exercise the new-user handling. One old user, Rahim, is given a full multi-stage attack that lines up with the sample network and SIEM reports.

replay_attack_samples.py — testing against real attacks

Replays a public collection of real recorded Windows attacks through the whole detection pipeline, treats each recording as one attack session, and reports how many were caught. This is the honest test: the model never trained on any of it.

nsf_kdd_benchmark.py — the academic benchmark

Runs the same Isolation Forest method on NSL-KDD, the standard public intrusion-detection dataset, reproducing the method used in the published literature so the results can be compared. It reports the overall scores and — because they are this thesis's focus — the detection rate for each attack category separately.

cert_train.py — learning normal from 1,000 real users

Reads the CERT r4.2 insider-threat dataset (Carnegie Mellon University) — a realistic record of a thousand users' logon, file and USB activity over months — turns each user-day into the eighteen-feature vector, and trains an Isolation Forest on it. This lets the model learn 'normal' from a large, realistic population rather than only the synthetic demo users. It states honestly which features CERT cannot fill (it has no process or authentication-failure logs).

cert_eval.py — measuring detection on real insiders

Evaluates the behavioural model on CERT using the dataset's own answer key: it labels every user-day that contains a real malicious event, trains on normal days only, and tests on held-out normals plus every malicious day. Because genuine insiders are only about 1.5% of the data, it reports the fair measures for rare positives — the ROC area and the share of insiders caught within a small review budget — rather than a misleading raw accuracy.

view_dataset.py, generate_code_doc.py, doc_content.py

Small helpers: print a dataset for inspection, and build this very document (the renderer and its text).

The data and output folders

Folder	What is inside
demo/	baseline_events.json (the 97 users' history) and current_events.json (today's session for all 100). This is the reproducible demo — the same result every time.
data/network/eve.json	A sample Suricata report: SSH scanning, a Cobalt Strike beacon, a malicious server, a 50 MB upload.
data/siem/alerts.json	A sample Wazuh report: SSH brute force, a switch to root, a rootkit warning.
data/macos/events.json	A sample macOS attack: a mail-borne AppleScript shell, a program from a USB volume, sudo, persistence.

data/datasets/	The real evidence: a public collection of recorded Windows attacks, and the NSL-KDD benchmark.
reports/	dashboard.html, the charts, metrics.json, and the record of the latest scan.
alerts/	One JSON file per alert, plus alerts.csv — the rolling log the dashboard reads.
database/	The small SQLite file of events and baselines, plus bookmarks recording how far each log has been read.
logs/	bads.log — the program's own detailed diary.
software/	windows-demo.exe (runs without Python), and the linux-demo / macos-demo launchers plus a script to build native binaries on those systems.

8. How to run the software

The easiest way

On Windows, double-click **Run-Demo.bat** (or `software\windows-demo.exe`). A menu appears:

```
[1] Own      - scan THIS computer once   (needs Administrator)
[2] Analyze  - report on the stored sample data
[3] Live     - keep monitoring THIS computer
```

- **Analyze** is the safe demonstration: it uses the 100 sample users and gives the same result every time. No special rights needed.
- **Own** scans this real machine once. It asks for Administrator, because reading the Windows Security log requires it. The first scan learns the machine's normal; later scans compare against it.
- **Live** repeats the scan on a timer and refreshes the dashboard by itself. Press Ctrl+C to stop.

From the command line

```
python bads.py analyze      # sample report
python bads.py own          # scan this machine
python bads.py watch        # continuous monitoring
python bads.py evaluate     # model scores + charts
python bads.py benchmark    # NSL-KDD benchmark
python bads.py dashboard --open
```

On Linux and macOS

```
chmod +x software/linux-demo
./software/linux-demo analyze    # sample report
sudo ./software/linux-demo own   # live scan (needs root to read
                                # auth.log / the unified log)
```

The Windows program is a single self-contained file that needs no Python. The Linux and macOS launchers run from the source and need Python installed; running `software/build-standalone.sh` on those systems produces a self-contained file for them too. (A program can only be packaged on the system it is built for — that is a limitation of the packaging tool, not of this project.)

What you will see

The terminal prints, for each person, the risk score, the machine-learning verdict, the decision, any attack chains found, the reasons, and the offending commands. Then the dashboard opens in the browser with the same information laid out visually.

9. Where the numbers come from (results)

Everything below is produced by the software itself and can be reproduced by running the commands shown.

The sample demonstration (100 users)

Command: *Analyze*. Result:

```
Monitored 100 users (97 known, 3 new) - 4 flagged, 96 clear
```

- **96 clear** — the ordinary users, each judged against their own habits, correctly left alone.
- **1 CRITICAL** — Rahim, the multi-stage attack: a USB stick at 2 a.m., a disguised program (invoice.pdf.exe) run from it, a scrambled PowerShell command, a payload in Downloads, credential theft, plus — from the network — an SSH scan, a Cobalt Strike beacon to a criminal server and a 50 MB upload, plus a Wazuh SIEM alert. Eleven correlation chains fired, including all four cross-plane ones.
- **3 REVIEW** — the brand-new users with no history yet: the system says so honestly rather than guessing.

Against real recorded attacks

Command: *replay_attack_samples*. 137 real published Windows attack recordings were replayed; **135 were flagged (98.5%)**. The two that were missed contain almost no evidence of the kind this system measures (one and three events, no command lines) — an honest limitation, not a hidden one.

The academic benchmark

Command: *benchmark*. The same Isolation Forest method on the standard NSL-KDD dataset:

Measure	Result
ROC area (threshold-independent)	0.944
Accuracy / F1	0.848 / 0.861
DoS detection	92.4%
Probe detection	99.9%
R2L detection	44.9%
U2R detection	73.1%

This result is the argument for the whole project, and it is worth reading carefully. Anomaly detection on network data alone is excellent at noisy attacks (DoS, scanning) and **weak at exactly the rare ones this thesis targets** — because at the network level R2L and U2R look like ordinary traffic. That is precisely why the system adds host behaviour, SIEM alerts and cross-plane correlation on top: to cover the gap the benchmark exposes.

An honest note on the demonstration figures

The evaluation on the project's own sample data scores very highly (F1 near 1.0). That is expected and should be stated plainly: the sample attacks contain clean, unambiguous signals. The trustworthy numbers are the two above — 98.5% on real recorded attacks, and 0.944 ROC area on the public benchmark.

In summary

The project is a small, honest security analyst made of software. It reads the computer's diary, adds meaning to each line using a knowledge base a human can edit, notices when several lines together tell an attack story, and separately learns what each machine's ordinary day looks like so it can spot a day that is not ordinary. It listens to two other professional tools — Suricata on the network and Wazuh on the logs — and trusts a finding most when two independent viewpoints agree. Then it explains itself: not just a verdict, but the reasons, the standard technique names, the exact commands, and which of the user's habits changed and by how much.

Every design choice in the code serves one of two goals: **catch the rare attacks that rule-based tools miss**, and **never cry wolf** — because a security tool nobody trusts is a security tool nobody uses.